Task & Feature Characterization for Multi-Class LLM Difficulty Routing

Aryansingh Sonaye
Student ID: AI25BTECH11032
Track: CostOps

October 2026

Abstract

Large Language Model (LLM) routing systems balance inference cost and performance by directing user queries to appropriate model tiers. A key bottleneck in conventional semantic routers is that prompt volume (length and token count) often artificially inflates the perceived difficulty of a task. In long-context software engineering workflows—where a prompt may contain thousands of tokens of repository documentation, build logs, and test suites surrounding a simple request—traditional routers frequently escalate easy tasks to expensive frontier models. This report documents a lightweight multi-signal extraction pipeline designed to decouple prompt length from task difficulty. By extracting eight rule-based structural features alongside standardized TF-IDF text semantics, the pipeline classifies incoming queries into three discrete routing tiers (`EASY`, `MEDIUM`, `HARD`). Evaluated on 5,475 held-out prompts from RouterBench, the system achieves **84.0% overall classification accuracy** and a **0.84 macro F1 score**, with **0.94 precision on HARD tier escalations**[cite: 8, 9].

1 Problem Statement & Motivation

1.1 The Context-Volume Distortion

Modern LLM routing gateways face two primary failure modes when estimating task difficulty:

1. **Long Prompt + Simple Task (`EASY`):** A developer submits a prompt containing 2,000 tokens of auxiliary code, stack traces, and documentation with the simple instruction: *"Fix the typo in the null-pointer exception."* Traditional embedding- or volume-based routers treat the huge token count as high difficulty, needlessly escalating the request to an expensive frontier model.

2. **Short Prompt + Complex Task (`HARD`):** A mathematician submits a brief 15-token query: *"Prove the generalized Poincaré conjecture for smooth 4-manifolds."* A size-based router misinterprets the short token length as trivial and routes it to an underpowered model.

1.2 CostOps Impact

In production environments, escalating long-but-easy prompts to frontier models scales inference costs linearly with token length. To achieve cost-efficient routing, a gateway must characterize intrinsic task complexity independently of auxiliary context volume.

2 Proposed Approach & Architecture

Rather than relying on high-latency LLM-as-a-judge calls or non-selective prompt compression (e.g., LLMLingua), our approach implements a **characterize-first, classify-second** paradigm executed in a single pass without model calls.

2.1 The 5 Signal Channels

The incoming prompt is decomposed across five parallel feature channels:

- **Context Volume (Amber Channel):** `token_count`, `cumulative_tokens` (measures total text scale).
- **Conversation Depth (Violet Channel):** `history_turns`, `multi_turn_flag` (captures follow-up dynamics).
- **Task Structure (Teal Channel):** `error_flag`, `code_present`, `ast_density` (detects code complexity and debugging).
- **Modality (Rose Channel):** `modality_flag` (detects non-text attachments or markdown images).
- **Semantics (Grey Channel):** TF-IDF vectorization capped at 1,000 uni- and bigrams to prevent text features from drowning out structural signals.

3 Implementation Details

3.1 Structural Signal Extraction Catalogue

All 8 structural signals are computed in $O(N)$ time over the prompt text:

Signal Name	Extraction Method / Regular Expression	Semantic Purpose
`token_count`	Whitespace split count of prompt string (min: 1)	Current prompt volume
`cumulative_tokens`	Running token count across all dialogue turns	Accumulated context
`history_turns`	Length of prompt turn array	Conversation depth
`multi_turn_flag`	Binary flag (1 if turns $>$ 1, else 0)	Follow-up refinement
`error_flag`	Regex: `Traceback`, `NullPointerException`, `TypeError`, `ValueError`, `AssertionError`, `RuntimeError`, `Exception`	Active debugging task
`code_present`	``` fence, or keywords: `def`, `class`, `import`, `return`, `function`, `const`	Code involvement
`modality_flag`	Image/video extensions (`.png`, `.jpg`, `.mp4`) or markdown `![]`	Non-text input
`ast_density`	Python AST nodes ÷ tokens on the first fenced block; 0.0 on failure	Structural code complexity

.

Table 1: Multi-signal feature extraction catalogue.

3.2 The AST Density Formulation

To prevent long documentation from artificially inflating structural complexity, **AST Density** measures structural nodes per token:

$$\text{AST Density} = \frac{\text{Count(Python AST Nodes in First Code Block)}}{\text{Total Prompt Token Count}}$$

If a 15-token function contains 20 AST nodes, its density is 1.33. If 1,350 words of documentation are added to that same snippet, the AST node count remains 20, but the density drops to 0.015. Thus, auxiliary context can dilute density, but can never artificially inflate it.

3.3 Model Pipeline & Feature Fusion

The classification pipeline is built using `scikit-learn`:

- **Feature Fusion:** `FeatureUnion` concatenating 1,000 TF-IDF features with the 8 structural signals passed through a `StandardScaler`.
- **Classifier:** Logistic Regression with L2 regularization ($C = 0.5$), balanced class weights, and max_iter = 1000.
- **Labels:** Processed from RouterBench raw data ($\approx$ 36.5k samples). Ground-truth integer labels (`0: HARD`, `1: MEDIUM`, `2: EASY`) were derived by calculating cost-weighted efficiency scores and binning into tertiles.

4 Experimental Results

Evaluated on a 15% held-out test split of **5,475 prompts** from RouterBench[cite: 8], the pipeline achieves an overall accuracy of **84.0%** and a macro F1-score of **0.84**[cite: 8].

Routing Tier	Precision	Recall	F1-Score	Test Support
`0: HARD`	**0.94**[cite: 9]	0.76[cite: 9]	0.84[cite: 9]	1,844[cite: 9]
`1: MEDIUM`	0.78[cite: 9]	**0.89**[cite: 9]	0.83[cite: 9]	1,805[cite: 9]
`2: EASY`	0.82[cite: 9]	0.87[cite: 9]	0.85[cite: 9]	1,826[cite: 9]

Table 2: Per-class classification metrics on the held-out test split.

Key Analytical Takeaways:

- **High HARD Precision (0.94):** When the router escalates a prompt to the `HARD` tier, it is correct 94% of the time, directly protecting compute budgets against false-positive model escalations[cite: 9].
- **HARD Recall (0.76):** Approximately 24% of genuinely complex prompts slip into `MEDIUM` or `EASY`, representing the primary target zone for future feature tuning[cite: 9].

5 Limitations & Edge Cases

1. **Vocabulary vs. Structure Bias:** In edge cases, domain-specific text vocabulary can outvote structural signals (e.g., short math prompts with academic jargon receiving inflated difficulty scores).
2. **Brittle Heuristic Extractors:** Regex keyword checks can occasionally trigger on prose discussions rather than active code.

3. **Language Scope for AST:** AST parsing is currently restricted to Python code blocks; non-Python languages fall back to 0.0 density.

4. **Proxy Labels:** RouterBench labels are derived from model cost-efficiency scores rather than human-annotated cognitive difficulty.

6 Future Work

1. **Context-Volume Ablation Study:** Retrain the pipeline with `token_count` and `cumulative_tokens` completely removed to empirically evaluate accuracy retention and confirm complete decoupling.

2. **Shared 4-Quadrant Benchmark:** Benchmark evaluation across the 2×2 matrix: [Short/Long $\times$ Easy/Hard].

3. **Latency Benchmarking:** Measure total feature extraction and linear inference latency to ensure compliance with sub-15ms gateway requirements.

4. **vLLM Semantic Router Integration:** Package the multi-signal extractor as a native custom feature provider for vLLM routing gateways.

7 Conclusion

The Multi-Signal Extraction pipeline demonstrates that lightweight, transparent structural signals combined with scaled text semantics offer a viable solution for difficulty routing. By isolating context volume into its own feature channel and capturing structural features like AST density, the system achieves an 84% accuracy score on RouterBench while keeping computational overhead minimal[cite: 8].